REPORT

All experiments were carried out on the university's linux20 and Linux15 machines, when there was no other user logged in

Number of cores : 4

Processor : Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

OS : NAME="Ubuntu" VERSION="20.04.6 LTS (Focal Fossa)"

Compiler Version : 9.4.0

Exercise 1.1

Threads	$n = 10^4$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0004	0.3766	0.3873	0.9766	97.276
2	0.0004	0.3787	0.1870	2.0233	101.23
4	0.0004	0.3788	0.1667	2.2733	56.843
8	0.0004	0.3750	0.1461	2.5700	32.150

Threads	$n = 10^5$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0044	34.012	38.194	0.9866	98.623
2	0.0065	34.080	17.871	1.9066	95.346
4	0.0072	34.051	13.627	2.5000	62.466
8	0.0104	34.069	10.688	3.1900	39.880

The problem of polynomial multiplication was solved as follows:

The original thread makes the rest of the threads and calculates the multiplication results, so that in the end we can check the validity of the parallel calculation result.

For the result, type int128_t is used so that there is no case of overflow.

Each thread calculates a number of coefficients, i.e.: the first thread could calculate the result x^7 but does all the operations that have to do with x^7. That is, x * x^6 ,x^2* x ^5... So we don't read all the results[i] in memory, as for example if each thread worked on the entire table of results. So there is no need for synchronization and there is
better cache locality since each thread works in a specific area of memory (relative to the results).

Initially in the times I noticed that the serial algorithm was slower than the parallel algorithm even with 1 thread.

This was due to the fact that in the serial algorithm we did

```
for(int i = 0; i <= degree; i++){
    for(int j = 0; j <= degree; j++){
      serial_result[i + j] += (  int128_t)poly1[i] * poly2[j];
    }
  }
```

Without a local SUM variable that can be kept in a register and be much faster and without limit control for which operations need to be done (see parallel algorithm).

In the end we changed it, and it is the same as the parallel algorithm because as we have said in the course we do not want to see an improvement due to a change of algorithm but a parallelization of the same algorithm. So the Results show that the acceleration increases with the number of threads. With 4 threads (as many as the physical cores), a speedup of ~2.3-2.5x is achieved with an efficiency of 57-62\%. With 8 threads, the speedup reaches ~2.6-3.2x but the performance drops to 32-40\%, as the extra threads share the same 4 natural cores, causing increased overhead from context switching.

Exercise 1.2

The problem of this exercise focuses on the management of a shared variable in a correct way so that no errors occur from the race condition. As stated in the speech, this request is implemented in three ways: use mutex, use read/write lock and use atomic operation. Each algorithm performs a for loop where it internally increases the shared variable after it has first provided the appropriate lock.

The times given are derived from the average of 5 executions.

Number of Reps: 10^6

Implementation	Thread count			
	1	2	4	8
mutex	.021598 seconds	.093251 seconds	.181114 seconds	.376882 seconds
Read-write lock	.027220 seconds	.170861 seconds	.395574 seconds	2.046468 seconds
Atomic operation	.005009 seconds	.038176 seconds	.081688 seconds	.163872 seconds

Number of reps: 10^7

Implementation	Thread count			
	1	2	4	8
mutex	.161587 seconds	.907191 seconds	1.906013 seconds	3.776430 seconds
Read-write lock	.283976 seconds	1.642358 seconds	4.221639 seconds	22.079834 seconds

Atomic operation	.061816 seconds	.323849 seconds	.825008 seconds	1.620213 seconds

The slowest implementation by far is the one using read/lock mutex, since our program only writes to the shared variable and does not take advantage of the ability to take the read lock from
multiple threads. Despite being used as a mutex, it is slower, since it probably has a more complex mechanism and different handling than that of mutex. The more the threads increase, the more obvious this cost is, since they are all writers and as a result the advantage of the read/write lock is not highlighted in this case.

Implementation with mutex shows a large increase in execution time with the increase of threads, since each thread needs to carry out
lock/unlock resulting in the majority waiting to present the lock.

Finally, atomic operations have the best performance with both the increase in threads and iterations and show smoother behavior than the other two. This is due to the fact that no one is being used
locking . Also, the choice of the memory order as relaxed partially affects the performance of the atomic operation, since it does not impose unnecessary restrictions.

Exercise 1.3

In this exercise we are asked to manage a shared struct containing 4 variables each of which refers to a table. These variables represent the number of non-zero elements of each table respectively and the purpose is to update them correctly in a serial and parallel way. The serial algorithm obviously runs through the arrays one by one and increases the corresponding variable each time it encounters non-
zero element. The parallel algorithm, on the other hand, assigns a table to each thread. The problem focuses on the management of the common structure, which however consists of four independent variables which are altered by the threads. With the implementation of the two cases, it appears that the two techniques produce the same results regardless of synchronization, i.e. it is not necessary. This is in line with our intuition, since threads don't actually change the same variable, so no wrong result can be obtained. In addition, the use of

locks increases the execution time since for each non-zero element encountered, the thread must provide the lock.

Below are the results obtained after 5 executions and the times are their average:

Algorithm	Table size		
	10^6	10^7	10^8
Serial	.126857 seconds	.127637 seconds	1.268830 seconds
Parallel	.142296 seconds	.132784 seconds	1.529656 seconds

We notice that from 10^7 to 10^8 array size there is a significant increase in time for the parallel algorithm, which is not expected intuitively, since the four threads work simultaneously and the parallel algorithm ends up being even slower even without the use of mutex. This is due to the cash line collisions that result from the different thread writes on the different structure variables that are in the same cache line. Specifically, this structure catches 32 bytes that are in the same cache line that is affected with each change applied to one of the four variables in the tables.

If we modify the structure so that each variable is in a different cache line, there is an acceleration for the parallel algorithm of about 70% (execution in .428783 seconds at 10^8 size)

Exercise 1.4

**The first 2 tables refer to pre-amendment times with additional functionality added to the balance questions.

Count of transactions = 10^7

Perc. of read (%)	Size of shared matrix							
	$n = 10^2$				$n = 10^4$			
	mutex-coarse	mutex-fine	rw-coarse	rw-fine	mutex-coarse	mutex-fine	rw-coarse	rw-fine
10	3.986	2.781	7.055	3.050	4.908	1.089	9.016	1.346
50	3.510	2.001	8.647	2.454	3.986	0.839	10.35	1.139
90	2.547	1.130	8.886	1.243	2.823	0.549	9.163	0.713

Count of transactions = $5 * 10^7$

Perc. of read (%)	Size of shared matrix							
	$n = 10^2$				$n = 10^4$			
	mutex-coarse	mutex-fine	rw-coarse	rw-fine	mutex-coarse	mutex-fine	rw-coarse	rw-fine
10	19.74	13.80	35.40	15.08	24.32	5.397	45.04	6.643
50	17.71	9.878	46.23	12.08	20.35	4.247	52.64	5.776
90	13.15	5.477	44.74	6.149	14.25	2.736	54.36	3.367

The last table refers to times after the amendment.

Count of transactions = 10^7

Perc. of read (%)	Size of shared matrix							
	$n = 10^2$				$n = 10^4$			
	mutex-coarse	mutex-fine	rw-coarse	rw-fine	mutex-coarse	mutex-fine	rw-coarse	rw-fine
10	6.241	3.167	8.860	3.388	7.041	1.404	10.62	1.635
50	16.43	3.795	23.55	3.792	16.61	2.459	25.46	2.708
90	28.65	4.312	18.48	3.803	28.49	3.421	18.38	3.490

The problem is as follows:

We have bank accounts and we want a fixed number of threads (we set 4 because if it was variable the total number would change transactions made at the bank and it would not be fair to compare a parallel)

to make 1) money transfers between accounts 2) account balance check. To avoid deadlocks, we always lock the accounts in ascending ID order. The exercise with coarse-grained locking (one mutex for the entire account structure) and fine-grained locking (one mutex per account) has been implemented. We also have read-write locks to allow multiple simultaneous readers (balance checks) but exclusive money transfers, again fine and coarse grained.

Initially we had made a mistake, we were calculating the number of reads and writes and we were running the reads together first and then all the transactions – writes. But this was a problem since transactions have to be done in a mixed way so that one read falls on a write so that for example rwlock makes sense. So ultimately each transaction is randomly selected within the iteration.

About the timings :

1) In general, it seems that as reads increase, time decreases. The end – Grained Mutex, the locks it makes are 1 for read and 2 for trade, so it makes sense that as the percentage of reads increases, the time decreases. In the In the case of Coarse Grained, only 1 mutex is locked but the work done in Write is more than in Read, so it makes sense to Time increases with more writes.

To our surprise at rwlock coarse grained **the** time increases along with the reads and searching we found a link in it

https://concurrencyfreaks.blogspot.com/2013/01/scalable-and-fast-read-write-locks.html

The following interest.

"When the computation done between the lock() and unlock() is lengthy enough, compared to the locking mechanism itself, then the contention on the read_counter is low and your application will not get a performance hit. If instead, the code that is done between the lock() and unlock() is short, then you run the risk of getting high contention on the read_counter, because you

have multiple threads trying to write and read to a single cache line, namely, the one where the read_counter is stored in memory."

So, the read section is very small (a simple access array), so the overhead of RW-lock covers the benefit of multiple readers. On the contrary, in fine grained rwlock the time with reads is reduced as expected.

2) Fine-grained locks are more efficient and this is explained especially when we have a larger number of bills since it is probably rarer 2 transactions to try to lock the same accounts. We can see this in the second table where for 10% reads with 100 accounts we have 19.74 Mutex coarse and 13.80 fine so speedup ~1.43x while for 10000 Accounts 24.32 for Coarse 5.397 for Fine Speedup ~4.5x. !

3) We also ran for a larger number of bills (100000) and noticed that there was nothing remarkable about it and we didn't put it on the table (increase of all times by 2-3 seconds since we have more cache misses).

4) We observe that when we quintuple transactions (from 10^7 to 5×10^7), the times increase approximately fivefold linearly so that the synchronization overhead remains constant regardless of the amount of work.

5) With 100 accounts and 10^7 transactions the time for fine grained logic is longer than with 10000 accounts and this results from the fact that with fewer accounts, the likelihood of try 2 threads to lock the same accounts so in the case of fine grained and the same mutex.

After adding a Loop to the read part of the transactions (we didn't put a mathematical function because it changed the times very little):

1) We see that the mutex coarse increases a lot, and in proportion to the increase in reads (before, the more reads multiplied, the less time decreased, now the more it increases).

2) Similar in fine grained mutex but to a much lesser extent (especially in 90% of reads the time taken up by each read lasts only 1 of 100-10000 mutexes and not 1 unique one)

3) In coarse rwlock we see again an increase in the time with reads because too many reads are delayed and writes are also delayed. Specifically, when we have 50-50 reads-writes, precisely because there are enough writes, so they are waiting for any
Time-consuming reads now, but there are also several reads. While in 90% reads the time is significantly reduced.

4) rwlock fine grained works as before but the time increases with the increase of reads instead of decreasing because now the critical section is more time-consuming (in 100 accounts the times are similar to the change in the percentage because there are several conflicts so fine grained does not have such a great benefit).

Exercise 1.5

This exercise examines the parallel synchronized execution of threads through a barrier, where each thread performs a for loop n iterations and in each iteration it has to wait for the rest of the threads to reach the same point in order to move forward. The three implementations use the pthread library barrier, a barrier with mutex and condition variable and a barrier with busy waiting. The above are quite different behaviors based on the number of threads and also iterations. The times given are derived from the average of 5 executions.

Number of reps: 10^5

Implementation	Thread count			
	1	2	4	8
Pthread barrier	.097267 seconds	.254771 seconds	.424490 seconds	.940293 seconds
Mutex – condition variable	.001955 seconds	.395310 seconds	.728269 seconds	1.756030 seconds
Busy waiting	.001984 seconds	.067570 seconds	.152138 seconds	1292.14478 seconds

Number of Reps: 10^6

Implementation	Thread count			
	1	2	4	8
Pthread barrier	.911826 seconds	2.519408 seconds	4.170159 seconds	9.345065 seconds
Mutex – condition variable	.019771 seconds	3.662619 seconds	7.268038 seconds	17.359694 seconds
Busy waiting	.018102 seconds	.652543 seconds	1.506999 seconds	

The execution time for busy-wait implementation for 8 threads increases dramatically compared to the times for threads as long as its cores system or less. (That's why I didn't take the average just for the
This is to be expected, since with busy waiting the cpu is constantly busy and there are constant context switches between threads unlike the other two methods where the threads wait and do not consume as many resources.

For the mutex-condition variable, the time increases a lot with the increase of threads, since locks have a high cost for such a simple implementation. The default barrier, on the contrary, presents the smoothest behavior because its internal implementation likely optimizes CPU usage depending on the wait times.

Regarding the Sense-reversal centralized barrier, the basic idea is that in each iteration all threads start with a local sense variable that initially has the same value as the global flag. Before entering the barrier they reverse the local sense, which always has the opposite value to the global flag at the moment the threads enter the barrier and then wait (via busy-waiting) while global flag != local sense. When the last thread reaches the barrier, it restores the appropriate variables for the new iteration and sets the flag = local sense, thus "unblocking" all the threads. In the next iteration, the flag will have the opposite value to the previous one, as well as the local sense

etc. That is, their values alternate in each iteration. This rotation ensures that no thread can leave the barrier of the next iteration without all the others having arrived. Even if one reached the barrier of the next iteration very quickly, it would have to be it is forced to busy-wait.